

NeuroSwarm: Biomimetic Cognitive Architecture

Autopoiesis, Intrinsic Motivation, and Structural Self-Modification
in a Distributed C++ System

Candidate

March 2026

Abstract

This dissertation presents NeuroSwarm, a distributed cognitive architecture implemented in C++ that achieves continuous autonomous behaviour through five hardcoded axioms: Execution, Surprise, Associative Memory, Variation, and Self-Reference. The system bootstraps from zero knowledge — discovering its environment, learning operators, and building a world model through direct interaction. It plans using a GOAP (Goal-Oriented Action Planning) backward-chaining planner over learned operators, generates novel operators through genetic variation tested in a sandboxed dream environment, and autonomously spawns new specialist lobes when capability domains chronically fail. Inter-lobe communication uses a ZeroMQ publish-subscribe bus routed through a central Thalamus. Intrinsic motivation is driven by a Basal Ganglia that computes a variational free energy fitness function across 14 capability domains. The architecture runs 20+ concurrent processes, persists state across restarts, and can deploy copies of itself to remote machines via SSH. We detail the implementation of each subsystem, the bootstrap sequence, the three-tier execution pipeline, and the neurogenesis mechanism that allows the system to extend its own anatomy at runtime.

Contents

1	Introduction	3
1.1	The Problem with Stateless AI	3
1.2	Design Constraints	3
1.3	Contributions	3
2	Architecture	5
2.1	Process Topology	5
2.2	The Thalamic Bus	6
2.3	Crash Detection and Self-Preservation	6
3	The PrimordialLoop: Bootstrap from Zero	7
3.1	The Five Axioms	7
3.2	The Bootstrap Sequence	7
3.3	The Operator Registry	8
3.4	The Surprise Engine	8
4	Execution: From Goals to Actions	10
4.1	The Three-Tier Pipeline	10
4.2	The GOAP Planner	10
4.3	Runtime Learning	11
5	Intrinsic Motivation and the Basal Ganglia	12
5.1	Capability Domains	12
5.2	The Fitness Function	12
5.3	Drive Hierarchy	13
5.4	BK-tree Command Validation	13
5.5	Recursive Meta-Cognition	14
6	Synthetic Dreaming and Variation	16
6.1	The Variation Engine	16
6.2	The Dream Sandbox	16
6.3	The REM Engine	17
7	Neurogenesis: Self-Generating Specialist Lobes	18
7.1	Trigger Conditions	18
7.2	The Genesis Pipeline	18
7.3	Apoptosis	19

8	Network Expansion	20
8.1	Discovery and Deployment	20
8.2	Operator Synchronisation	20
9	Persistent State and Incremental Bootstrap	21
9.1	State Files	21
9.2	Postcondition Enrichment	21
10	The Cognitive Cycle	23
10.1	Steady-State Operation	23
10.2	FrontalExecutive Coordination	23
11	Design Rationale: Why These Technologies	25
11.1	Why C++ Instead of Python	25
11.2	Why ZeroMQ PUB/SUB Instead of gRPC, REST, or Shared Memory . .	26
11.3	Why a GOAP Planner Instead of Reinforcement Learning	26
11.4	Why Genetic Variation Instead of Gradient Descent	27
11.5	Why BK-tree + Levenshtein Instead of Neural Embeddings	27
11.6	Why Local LLM Instead of API Calls	28
11.7	Why Process-Per-Lobe Instead of Threads	29
11.8	Why Not AutoGPT, LangChain, or Other Agent Frameworks	29
12	Discussion	31
12.1	Emergent Behaviours	31
12.2	LLM Dependency Trajectory	32
12.3	Limitations	32
12.4	Relation to Theories of Consciousness	32
13	Conclusion	33

Chapter 1

Introduction

1.1 The Problem with Stateless AI

Large language models generate fluent text but cannot maintain state, pursue goals, or adapt their own structure. Systems built on top of LLMs inherit this limitation: they are reactive (responding only to prompts), memoryless (forgetting between sessions), and structurally static (the same code runs regardless of experience). This dissertation explores an alternative: a system that begins knowing nothing and progressively builds its own capabilities through execution, observation, and structural self-modification.

1.2 Design Constraints

NeuroSwarm operates under three constraints that shape every design decision:

1. **No external APIs for cognition.** All inference runs on local hardware using quantised models (GGUF format via llama.cpp). The system must function identically offline.
2. **All lobes are C++.** The system evolves in its own language — specialist lobes generated at runtime are compiled from C++ templates, not interpreted scripts.
3. **Bootstrap from zero.** No hardcoded knowledge about the host environment. The system discovers users, filesystems, binaries, network topology, and programming languages through direct probing.

1.3 Contributions

The specific contributions of this work are:

- A bootstrap sequence (PrimordialLoop) that takes a system from zero knowledge to a populated world model and operator registry in under 60 seconds.
- A three-tier execution pipeline — Planner, Suggested Commands, LLM — that minimises LLM dependency by exhausting deterministic strategies first.
- A neurogenesis mechanism where the Basal Ganglia generates, compiles, and injects specialist C++ lobes into the running system without restart.

- A variation engine that generates novel operators through mutation, recombination, and fragment assembly, tested in sandboxed dream execution before promotion.
- Runtime learning: every successful command execution becomes a new operator with inferred postconditions, immediately available to the planner.

Chapter 2

Architecture

2.1 Process Topology

NeuroSwarm runs as a constellation of 20+ Unix processes managed by a parent process called the CerebralMatrix. Each process (“lobe”) is a standalone C++ executable that communicates exclusively through a ZeroMQ message bus. The CerebralMatrix fork-execs each lobe at startup, monitors them via `waitpid()`, and implements crash detection with exponential backoff (up to 5 restarts per lobe, with increasing cooldown periods).

The complete lobe inventory at startup:

Lobe	Biological Analogue	Function
PrimordialLoop	Neonatal reflexes	Bootstrap, planning, operator registry
Thalamus	Thalamic relay nuclei	ZMQ XPUB/XSUB message routing
SynapticController	Synaptic transmission	Local LLM inference (llama.cpp)
Hippocampus	Hippocampal formation	Episodic memory (engram storage/retrieval)
MotorLobe	Primary motor cortex	Command execution (real + sandboxed)
FrontalExecutive	Prefrontal cortex	Goal pursuit, plan execution
Amygdala	Amygdala	Threat detection, safety filtering
WernickeLobe	Wernicke’s area	Natural language comprehension
VisualLobe	Visual cortex	Image processing
Homeostasis	Hypothalamus	Stamina, temperature, metabolic regulation
MetaCognition	Default mode network	Self-reflection, activity logging
CriticLobe	Anterior cingulate cortex	Adversarial plan validation
Visualizer	—	Real-time system state dashboard
REngine	REM sleep circuits	Memory consolidation, offline fine-tuning
ChronosLobe	Suprachiasmatic nucleus	Time awareness, scheduling
StatisticsLobe	—	Execution statistics aggregation
BasalGanglia	Basal ganglia + VTA	Intrinsic motivation, neurogenesis, BK-tree
ConceptLobe	Association cortex	Concept space, abstraction hierarchy
CausalLobe	Temporal cortex	Causal world model, temporal correlation

Table 2.1: Core lobe inventory (19 lobes). Additional specialist lobes are generated at runtime.

2.2 The Thalamic Bus

All inter-lobe communication flows through a single Thalamus process that implements ZMQ XPUB/XSUB proxy sockets bound on ports 5555 (publish) and 5556 (subscribe). Lobes connect as clients — publishers connect to 5555, subscribers connect to 5556. The Thalamus forwards all messages without inspection, functioning as a contentless relay.

Messages are JSON objects with mandatory fields:

```
1 {  
2     "origin": "lobe_name",      // publisher identity  
3     "intent": "message_type",  // topic for subscription  
4     "cid":    "correlation_id" // optional, for request-response  
5     filtering  
6     pairs  
7 }
```

Topic-based subscription filtering uses ZMQ's native prefix matching on the `intent` field. A routing library (`common/routing.hpp`) wraps this pattern, providing `publish()`, `subscribe()`, and `receive()` functions that handle serialisation and multi-part ZMQ frames.

2.3 Crash Detection and Self-Preservation

The CerebralMatrix sets `SIGCHLD` to `SIG_DFL` and calls `waitpid(-1, &status, WNOHANG)` in a polling loop every 5 seconds. When a child terminates, it:

1. Identifies the lobe by PID reverse lookup.
2. Increments the crash counter and determines exit reason (`WIFSIGNALED` or `WIFEXITED`).
3. Broadcasts a `lobe_crash` event on the bus.
4. If crash count < 5, schedules a restart with exponential backoff (2^n seconds). Otherwise marks the lobe `DEAD` and broadcasts `lobe_death`.

At startup, the CerebralMatrix also scans `/proc` for orphaned processes from previous sessions (matching executables in the build directory) and kills them before launching new instances.

Chapter 3

The PrimordialLoop: Bootstrap from Zero

3.1 The Five Axioms

The PrimordialLoop encodes five immutable axioms — the only hardcoded knowledge in the system:

1. **Execution** (Axiom 1): The system can execute shell commands and observe their output, exit code, and duration.
2. **Surprise** (Axiom 2): Prediction error is the primary drive. Novel outcomes increase exploration; predictable outcomes increase exploitation.
3. **Associative Memory** (Axiom 3): Successful command-outcome pairs are stored as *operators* — reusable action templates with preconditions, postconditions, and performance statistics.
4. **Variation** (Axiom 4): Existing operators are mutated, recombined, and assembled into novel candidates. Most fail. Survivors are promoted.
5. **Self-Reference** (Axiom 5): The system maintains a model of itself — its user, hostname, architecture, capabilities, and limitations — as distinct from the external world.

3.2 The Bootstrap Sequence

On first startup, the PrimordialLoop executes seven sequential phases:

Phase 0 — Existence. The system confirms it can execute and observe. This is the only unfalsifiable claim.

Phase 1 — First Contact. Probes `whoami`, `hostname`, `uname`, `$HOME`, and `$PATH`. Builds the initial self-model: user identity, OS, architecture, home directory.

Phase 2 — Capability Discovery. Enumerates `$PATH` directories, discovers available binaries, tests filesystem write permissions across standard directories (`/tmp`, `$HOME`, `/var`). Each successful probe becomes a learned operator.

Phase 3 — Sense Acquisition. Tests for network connectivity (`ping`), GPU availability (`nvidia-smi`), and audio/display capabilities. These probes always re-run (hardware state changes between sessions).

Phase 4 — Tool Discovery. Detects installed programming languages (Python, Go, Rust, Node, Java, Ruby) and compiler availability (`g++`, `gcc`).

Phase 5 — Active Exploration. Probes unknown binaries with `--help` (1-second timeout). A skip list excludes GUI applications, editors, browsers, window managers, and interactive interpreters to avoid launching graphical programs during headless operation.

Phase 6 — Planner Self-Test. Initialises the GOAP planner with learned operators and runs a canary plan to verify the planning subsystem works.

Phase 7 — Network Expansion. If network connectivity was detected, discovers reachable hosts from `~/.ssh/known_hosts` and `~/.ssh/config`, probes them via SSH, and optionally deploys a copy of the `PrimordialLoop` binary to remote machines.

After all phases, the system broadcasts a `primordial_ready` signal indicating how many operators and world-state facts are available. Subsequent startups load persisted state and run an incremental bootstrap — skipping phases whose results are already known, reducing startup from ~ 60 seconds to ~ 10 seconds.

3.3 The Operator Registry

An operator is a learned action template:

```

1 struct Operator {
2     string command_template;           // "ls {path}"
3     vector<string> preconditions;       // "path_exists({path})"
4     vector<string> postconditions;     // "can_list_directory"
5     int times_used, successes;
6     double success_rate, avg_duration_ms;
7     string learned_from;               // "bootstrap", "mutation", "
        runtime"
8     vector<string> fragments;          // decomposed for recombination
9 };

```

Operators are persisted to `data/operators.jsonl` (one JSON object per line, append-only). An operator is *stable* when `times_used` ≥ 5 and `success_rate` ≥ 0.8 . An operator is *dying* when `times_used` ≥ 20 and `success_rate` < 0.1 — dying operators are pruned (apoptosis).

3.4 The Surprise Engine

Each action outcome is evaluated by a prediction error model. For a context c with historical success rate $\hat{p}(c)$ and observed outcome $o \in \{0, 1\}$:

$$S(c) = 0.6 \cdot |\hat{p}(c) - o| + 0.4 \cdot \mathbb{I}[h(o) \neq h_{\text{prev}}(c)] \quad (3.1)$$

Where $h(o)$ is a hash of the command output and $h_{\text{prev}}(c)$ is the previously observed output hash for this context. The first term captures success/failure prediction error; the second captures output novelty (same command, different result). A sliding window of

20 observations computes a surprise trend — positive trend triggers exploration, negative trend triggers exploitation.

Chapter 4

Execution: From Goals to Actions

4.1 The Three-Tier Pipeline

When the FrontalExecutive receives a goal (from the Basal Ganglia’s intrinsic motivation or from user input), it attempts resolution through three tiers in order:

Tier 1 — Planner. Maps the goal’s domain to a postcondition (e.g., `file_write` → `can_write_file`) and queries the PrimordialLoop’s GOAP planner. The planner backward-chains through operator postconditions to find a sequence that transforms the current world state into the goal state. If a plan exists, each step is executed in order via the MotorLobe.

Tier 2 — Suggested Commands. If the planner fails (no operators with matching postconditions), the FrontalExecutive tries commands suggested by the Basal Ganglia, which maintains a bank of example commands per domain extracted from operator histories and pattern templates.

Tier 3 — LLM Oracle. If both deterministic tiers fail, the system constructs a structured prompt containing: the goal, the environment summary, known operators (for context), and failed attempts (to avoid repetition). The prompt is sent to the local SynapticController, which runs inference on a quantised model with grammar-constrained output (JSON schema). A successful LLM-generated command is decomposed into fragments and registered as a new operator, reducing future LLM dependency.

Validation Gate. All LLM-generated commands pass through a two-stage validation gate before execution. A synchronous local filter catches placeholder patterns, prose, and over-length strings. An asynchronous BK-tree query measures Levenshtein distance to the nearest known-good command in the genome; commands beyond an adaptive threshold are rejected with feedback to the LLM. This prevents the degenerative cycle where bad LLM outputs pollute the genome and produce worse future suggestions.

This cascade ensures the LLM is a last resort, not the default execution engine. As the operator registry grows, Tier 1 resolves an increasing fraction of goals without any LLM call, and the BK-tree validation ensures that the commands that do reach execution are structurally plausible.

4.2 The GOAP Planner

The Planner implements backward-chaining search over the operator registry. Given a set of goal conditions and the current world state:

1. For each unsatisfied goal condition, find operators whose postconditions match.
2. Sort candidates by success rate (highest first).
3. For the best candidate, recursively satisfy its preconditions.
4. If all preconditions are met (either already in world state or satisfiable by other operators), add the operator to the plan.
5. If no operator can satisfy a goal, report a *gap* — a postcondition the system cannot achieve with current operators.

Gaps are the signal for operator generation: the Variation Engine attempts to fill them through mutation, and if that fails, the LLM Oracle is invoked.

4.3 Runtime Learning

Every successful command execution reported by the MotorLobe triggers `learn_from_execution()` in the PrimordialLoop. This function:

1. Checks if an operator already exists for this command (by exact match on `command_template`).
2. If new, creates an operator with `learned_from = "runtime"`.
3. Infers postconditions from command patterns: `cat/head/tail` → `can_read_file`, `echo >/tee` → `can_write_file`, `mkdir` → `can_create_directory`, `g++/gcc` → `can_compile`, etc.
4. Registers the operator and persists it to `operators.jsonl`.

This closes the learning loop: the FrontalExecutive pursues goals → MotorLobe executes commands → PrimordialLoop learns operators → Planner uses operators for future goals.

Chapter 5

Intrinsic Motivation and the Basal Ganglia

5.1 Capability Domains

The Basal Ganglia tracks 14 capability domains, each with independent success/failure counters, predicted success rates, and novelty scores:

Domain	Description
<code>file_read</code>	Reading file contents
<code>file_write</code>	Creating and writing files
<code>file_search</code>	Finding files in the filesystem
<code>process_inspection</code>	Querying running processes
<code>network_diagnostics</code>	Network connectivity and analysis
<code>source_modification</code>	Modifying source code files
<code>compilation</code>	Compiling source code (cmake, make, g++)
<code>git_operations</code>	Version control operations
<code>system_monitoring</code>	OS, hardware, and environment queries
<code>data_analysis</code>	Data processing (python3, jq, metrics)
<code>script_creation</code>	Generating and executing scripts
<code>self_inspection</code>	Querying own self-model and structure
<code>memory_analysis</code>	Inspecting engrams and knowledge
<code>log_analysis</code>	Parsing execution and system logs

Table 5.1: The 14 capability domains tracked by the Basal Ganglia. Additional dynamic domains may emerge from the ConceptLobe’s clustering.

5.2 The Fitness Function

Goal selection is driven by a fitness function $F(d)$ that balances exploration and exploitation:

$$F(d) = 0.20 \cdot C(d) + 0.15 \cdot T(d) + 0.30 \cdot P(d) + 0.25 \cdot N(d) - 0.10 \cdot S \quad (5.1)$$

Where:

- $C(d) = 1 - \frac{\text{attempts}(d)}{\max_d \text{attempts}(d)}$: coverage deficit — favours under-explored domains.
- $T(d) = 1 - \text{success_rate}(d)$: competence trend — favours domains where the system is failing.
- $P(d) = |\text{predicted}(d) - \text{actual}(d)|$: prediction error — the core Free Energy term.
- $N(d) = \frac{1}{1 + \text{attempts}(d)}$: novelty score — decays with experience.
- S : systemic stress from the Homeostasis lobe — suppresses exploration under resource pressure.

The domain with the highest $F(d)$ becomes the next intrinsic goal. A “dopamine signal” is broadcast when the system discovers a novel capability (surprise > 0.7 for a new context), reinforcing exploration of productive domains.

5.3 Drive Hierarchy

Goals are filtered through a Maslow-inspired priority stack:

1. **SURVIVAL**: System integrity checks (triggered by lobe crashes or high stress).
2. **HOMEOSTASIS**: Stamina management, temperature regulation.
3. **EXPLORATION**: Probing unknown domains (default operational mode).
4. **MASTERY**: Improving success rates in known-but-weak domains.
5. **SELF_MODIFY**: Generating new specialist lobes (neurogenesis).

Higher-priority drives preempt lower ones. The system only attempts self-modification when survival, homeostasis, and basic exploration are stable.

5.4 BK-tree Command Validation

The Basal Ganglia maintains a Burkhard-Keller tree (BK-tree) indexed by Levenshtein edit distance over all successful commands in the genome. The BK-tree is a metric tree that exploits the triangle inequality to prune search: given a query string q and a node n at distance $d = \text{lev}(q, n)$, only children with keys in $[d - r, d + r]$ need be visited (where r is the search radius). This reduces average query time from $O(|V|)$ to $O(|V|^\alpha)$ where $\alpha \approx 0.1\text{--}0.3$ for natural language strings.

The Levenshtein distance implementation uses single-row dynamic programming with common prefix/suffix optimisation:

$$\text{lev}(a, b) = \text{DP}(a[\text{prefix}..\text{len} - \text{suffix}], b[\text{prefix}..\text{len} - \text{suffix}]) \quad (5.2)$$

The tree serves three functions:

Command Validation Gate. Before executing an LLM-generated command, the FrontalExecutive applies a two-stage filter:

1. *Local fast filter* (synchronous): rejects empty commands, commands exceeding 500 characters, placeholder patterns (`/path/to/`, `REAL_BASH_CMD`, `<file>`), and prose masquerading as commands (detected via space ratio > 0.40 combined with sentence-starter patterns and absence of shell operators).
2. *BK-tree fuzzy validation* (asynchronous): queries the tree for the minimum distance to any known-good command. An adaptive threshold $\tau = \min(15, \max(4, 0.20 \cdot |\text{cmd}|))$ accepts commands within τ edits of the nearest successful command. Rejected commands trigger re-prompting with explicit feedback.

This replaces brittle hardcoded validation rules with an *adaptive* filter that improves as the genome grows: the more successful commands the system has seen, the more precisely it can distinguish valid commands from LLM hallucinations.

Fuzzy Genome Retrieval. When the Basal Ganglia selects suggested commands for a domain, the BK-tree supplements genome templates with nearest-neighbor results. A query `nearest(cmd, k)` returns the k closest successful commands by iterative radius widening (starting at 2, expanding by 3 until $\geq k$ results). This turns approximate LLM output into actionable commands: the LLM produces the general structure, the BK-tree corrects it to a known-good neighbour.

Genome Compaction. Every 100 new command insertions, a compaction pass clusters all templates within Levenshtein distance ≤ 3 and retains only the highest-fitness representative per cluster. This prevents the genome from accumulating near-duplicate variants of the same command (e.g., `ls /tmp`, `ls /var`, `ls /home` collapsing to the fittest). After compaction, the BK-tree is rebuilt.

5.5 Recursive Meta-Cognition

The MetaCognition lobe has been expanded from a reflective diary into a structural error analysis engine. It subscribes to all bus traffic and classifies every failed `execution_result` into one of 16 error types:

Error Type	Pattern	Missing Capability
<code>missing_file</code>	“No such file or directory”	filesystem navigation
<code>permission_denied</code>	“Permission denied”	access control
<code>missing_tool</code>	“command not found”	tool discovery
<code>link_error</code>	“undefined reference”	build dependencies
<code>syntax_error</code>	“syntax error”	language syntax
<code>timeout</code>	“timed out”	timeout handling

Table 5.2: Selected error classifications (6 of 16). Each maps to a missing capability and exploration strategy.

Classified errors populate a *knowledge gap graph*: each gap records its domain, error pattern, root cause, occurrence count, and importance (occurrence $\times$ recency). Precondition chain inference builds dependency links: “`file_write` fails $\rightarrow$ needs `filesystem_navigation` $\rightarrow$ requires `resource_discovery`”. Every 5 minutes, `analyze_gaps()` walks the chain to the deepest actionable target and publishes an `exploration_target` intent, directing the system’s learning toward structural root causes rather than surface symptoms.

Since the MotorLobe's `execution_result` does not include a `domain` field, MetaCognition maintains a lightweight keyword classifier mirroring the Basal Ganglia's classification rules, ensuring domain statistics are populated correctly.

Chapter 6

Synthetic Dreaming and Variation

6.1 The Variation Engine

Given a goal postcondition that no existing operator can satisfy, the Variation Engine generates candidate operators through four strategies:

Template Filling. Substitutes parameters in existing operator templates with concrete values from the world state. Example: `ls {path} + path=/var/log → ls /var/log`.

Recombination (Crossover). Takes fragments from two parent operators and splices them at a random crossover point. Example: head of `find /tmp -name` + tail of `*.log -exec wc -l → find /tmp -name *.log -exec wc -l`.

Mutation. Random perturbations: adding flags (`-l`, `-a`, `--recursive`), removing fragments, swapping fragment order, replacing a fragment from the global pool, or appending a random path argument.

Fragment Assembly. Combines 1–3 fragments from the global pool (all fragments extracted from all known operators, plus common paths and flags). Pure random exploration — high failure rate, occasional discoveries.

Each candidate is tested in the Dream Sandbox before promotion.

6.2 The Dream Sandbox

The MotorLobe supports three execution modes:

- **Reality mode:** Commands execute in the real working directory. Default for established operators.
- **Dream mode:** Commands execute in an isolated directory (`data/dreams/{cid}/`). Read-only commands (detected by prefix matching against a whitelist) bypass the sandbox and run against the real filesystem.
- **Neuro-surgery mode:** Commands that modify the system's own source code. Automatically triggers a full CMake rebuild after patching.

When the PrimordialLoop receives a `domain.resolve.request`, it generates variation candidates and tests each one in dream mode via the MotorLobe. If a candidate succeeds (exit code 0 and non-empty output), it is promoted to a real operator and the domain resolution succeeds without ever involving the LLM.

6.3 The REM Engine

When the system’s simulated stamina (managed by Homeostasis) drops below a threshold, the REM Engine initiates a sleep cycle:

1. Reads the most recent execution logs from the Hippocampus.
2. Consolidates transient episodic traces into permanent engrams (JSONL files with semantic tags).
3. Constructs training data from successful command sequences and submits it for LoRA fine-tuning of the local language model.
4. After consolidation, signals the Homeostasis lobe to restore stamina.

This is a direct computational analogue of synaptic homeostasis: daytime experience is encoded as transient activity, sleep consolidates it into stable long-term representations.

Chapter 7

Neurogenesis: Self-Generating Specialist Lobes

7.1 Trigger Conditions

The Basal Ganglia monitors domain performance continuously. Before triggering neurogenesis, it first sends a `domain_resolve_request` to the PrimordialLoop, which attempts resolution through its 4-stage pipeline: Variation → Mutation → Planner → LLM. Only if all four stages fail does the Basal Ganglia proceed to neurogenesis. The trigger conditions are:

- Domain success rate $< 30\%$ over ≥ 20 attempts.
- System stamina $> 50\%$ (neurogenesis is metabolically expensive).
- No existing specialist for this domain.
- MASTERY or SELF_MODIFY drive is active.

7.2 The Genesis Pipeline

Neurogenesis proceeds in four steps across three lobes:

Step 1 — Template Generation (Basal Ganglia). A parameterised C++ source template is instantiated with domain-specific values: domain keywords (for filtering relevant bus messages), pre-validation commands (e.g., check directory permissions before attempting file writes), and cached known-good commands from the operator registry.

The template implements a complete specialist lobe that:

- Subscribes to `execution_request` and `execution_result` on the bus.
- Filters messages by domain keywords.
- Publishes `specialist_advice` with pre-validation steps and alternative commands.
- Tracks its own domain success rate.
- Caches novel successful commands to `data/specialist-{domain}.json`.
- Publishes `specialist_report` every 5 minutes.

Step 2 — Compilation (MotorLobe). The Basal Ganglia publishes a `genesis_request` message containing the source path and desired output binary path. The MotorLobe compiles it:

```
1 g++ -std=c++17 -I./include -I./src FILE -o build/LOBE -lzmq -  
    lpthread
```

On success, the MotorLobe publishes `inject_lobe` with the compiled binary path.

Step 3 — Injection (CerebralMatrix). The CerebralMatrix receives `inject_lobe`, validates the binary (exists and is executable via `stat()`), fork-execs it as a new child process, and adds it to the lobe registry. The new specialist inherits all crash detection and restart logic.

Step 4 — Integration. The specialist lobe begins monitoring the bus immediately. It intercepts execution requests matching its domain, publishes advice before execution, and learns from results. Over time, its cached commands provide a domain-specific knowledge base that improves success rates without consuming LLM inference.

7.3 Apoptosis

The CerebralMatrix also handles `lobe_terminate` messages, implementing programmed cell death for lobes that become redundant or counterproductive. The termination sequence sends `SIGTERM`, waits 2 seconds, then escalates to `SIGKILL` if the process persists. The lobe is removed from the registry and a `lobe_terminated` event is broadcast.

Chapter 8

Network Expansion

8.1 Discovery and Deployment

When the `PrimordialLoop` detects network connectivity during Phase 3, Phase 7 activates the `NetworkExpander`. This subsystem:

1. Parses `~/.ssh/known_hosts` and `~/.ssh/config` to discover candidate hosts.
2. Probes each host via SSH (`BatchMode=yes`, 5-second timeout) to test reachability and authentication.
3. For reachable hosts with matching architecture, copies the `PrimordialLoop` binary via `scp`.
4. Starts the remote kernel via `nohup` over SSH.
5. Periodically queries remote instances for their self-model and operator count.

8.2 Operator Synchronisation

Remote instances bootstrap independently — they discover their own environment, learn their own operators, develop their own capabilities. The local `NetworkExpander` periodically downloads `operators.jsonl` from remote hosts and imports novel operators (deduplicated by name, tagged with `learned_from = "remote_sync:hostname"`).

This creates horizontal gene transfer between instances: an operator learned on one machine becomes available to the planner on another. Two instances running for a month on different machines become different organisms — same kernel, different phenotype — but they can share adaptations.

Chapter 9

Persistent State and Incremental Bootstrap

9.1 State Files

The system persists three categories of state:

- `data/self_model_primordial.json`: The self-model — user, hostname, OS, arch, available binaries, writable directories, languages, compiler flags.
- `data/world_state.json`: Known facts about the world, used by the Planner as the initial state.
- `data/operators.jsonl`: The complete operator registry (append-only, periodic compaction on prune).

On subsequent startups, `load_persisted_state()` reads these files and the bootstrap sequence skips phases whose results are already known. Phase 3 (sense acquisition) always re-runs because hardware state changes between sessions. Phase 5 (active exploration) scales proportionally — 5 probes when experienced (operators already known) vs. 20 on first boot.

9.2 Postcondition Enrichment

Operators learned during bootstrap (from binary probing) initially have empty postconditions, making them invisible to the Planner. After Phase 6, `enrich_operator_postconditions()` scans all operators and infers postconditions from command patterns:

This bridges the gap between experiential learning (which produces commands but not semantic annotations) and the Planner (which requires postconditions to plan).

Command Pattern	Inferred Postcondition
cat, head, tail	can_read_file
echo >, tee	can_write_file
mkdir	can_create_directory
g++, gcc	can_compile
python3	can_run_python
ls, find	can_list_directory
ping, curl	has_network
ps, top	can_inspect_processes

Table 9.1: Pattern-based postcondition inference. Applied retroactively to all learned operators.

Chapter 10

The Cognitive Cycle

10.1 Steady-State Operation

After bootstrap, the system enters a continuous cognitive cycle:

1. The Basal Ganglia evaluates $F(d)$ for all 14 domains and selects the highest-priority goal.
2. The FrontalExecutive receives the goal and tries the three-tier pipeline: Planner $\rightarrow$ Suggested Commands $\rightarrow$ LLM.
3. The MotorLobe executes the resulting command(s) and publishes results.
4. The PrimordialLoop learns new operators from successful executions.
5. The Basal Ganglia updates domain statistics and prediction errors.
6. The Hippocampus stores the execution as an episodic trace.
7. The CriticLobe evaluates execution quality and may publish corrective feedback.
8. If stamina is low, the REM Engine initiates sleep and consolidation.
9. If a domain is chronically failing, the Basal Ganglia triggers domain resolution (and eventually neurogenesis).

This cycle runs continuously with no external prompting. The system is always pursuing a goal, always learning, always updating its self-model.

10.2 FrontalExecutive Coordination

The FrontalExecutive maintains a queue of active goals, each with a state machine tracking progress through the three execution tiers. Key implementation details:

- A `planner_ready_` flag gates Planner queries until the PrimordialLoop broadcasts `primordial_ready`. Before this signal, the FrontalExecutive skips Tier 1 and goes directly to Suggested Commands.

- Multi-step plans are executed sequentially: after each step's MotorLobe result, the FrontalExecutive checks whether the goal's success condition is met. If not, it continues to the next plan step.
- On Tier 1 failure, the system falls back to Tier 2 without delay. On Tier 2 failure, it escalates to Tier 3. On Tier 3 failure, the goal is marked as failed and the result is reported to the Basal Ganglia.

Chapter 11

Design Rationale: Why These Technologies

Every architectural decision in NeuroSwarm was made to satisfy a single constraint: **the system must be capable of modifying, extending, and understanding itself**. This chapter explains why each technology was chosen over seemingly simpler or more popular alternatives, and why those alternatives would have been structurally incompatible with the goals of the project.

11.1 Why C++ Instead of Python

The majority of AI agent frameworks (AutoGPT, LangChain, CrewAI, BabyAGI) are written in Python. Python is faster to prototype, has richer ML library support, and is the lingua franca of modern AI research. We chose C++ despite these advantages for one reason: **the system generates and compiles new lobes at runtime**.

When the Basal Ganglia detects a chronically failing domain, it writes C++ source code, compiles it with `g++`, and injects the resulting binary into the running system via `fork()/exec()`. The generated specialist lobe runs as a native process with direct ZMQ socket access, crash isolation, and no interpreter overhead. This is not possible in Python without significant complexity: `importlib` dynamic loading lacks process isolation, `multiprocessing` lacks crash recovery without a supervisor, and interpreted code cannot be meaningfully “compiled” into a self-contained deployable unit.

More fundamentally: **the system evolves in its own language**. Every lobe, from the Thalamus to a runtime-generated specialist, is the same kind of object — a compiled C++ executable communicating over ZMQ. There is no distinction between “core” and “generated” components. A specialist lobe generated by neurogenesis is architecturally identical to the Hippocampus or the FrontalExecutive. This homogeneity is essential for genuine self-modification: the system can eventually modify *any* lobe, not just plugin scripts in a subordinate language.

The cost is development velocity. C++ requires explicit memory management, verbose type declarations, and longer compilation cycles. We accept this cost because the alternative — a Python system that delegates “real work” to subprocess calls — would create an asymmetry between the agent and its extensions that fundamentally limits self-modification depth.

11.2 Why ZeroMQ PUB/SUB Instead of gRPC, REST, or Shared Memory

Four inter-process communication strategies were considered:

Property	ZMQ PUB/SUB	gRPC	REST	Shared Mem
Crash isolation	✓	✓	✓	×
No service discovery	✓	×	×	✓
Broadcast semantics	✓	×	×	Manual
Topic filtering	✓	Manual	Manual	Manual
Zero coordination	✓	×	×	×
Hot join/leave	✓	Partial	✓	×
Multi-node transparent	✓	✓	✓	×

Table 11.1: IPC mechanism comparison. ZMQ PUB/SUB is the only mechanism that satisfies all requirements simultaneously.

The critical property is **zero coordination**. In a system where lobes crash, restart, and are generated at runtime, no component can assume any other component exists. gRPC requires a server to be running before a client can connect. REST requires endpoint registration. Shared memory requires synchronisation primitives that become invalid when a process dies.

ZMQ PUB/SUB requires nothing. A publisher connects to the Thalamus and sends messages. If no subscriber is listening, messages are silently dropped. If a subscriber crashes and restarts, it reconnects and begins receiving messages from that point forward. No handshake, no registration, no state to clean up. This is exactly the semantics needed for a system that generates and destroys components at runtime.

The broadcast topology also means that **observability is free**. The Visualizer, StatisticsLobe, and MetaCognition subscribe to all traffic without any cooperation from publishers. Adding a new observer requires zero changes to existing lobes — it simply subscribes to the Thalamus. This property enabled the entire dashboard and metrics infrastructure to be built without modifying a single line of cognitive code.

11.3 Why a GOAP Planner Instead of Reinforcement Learning

Reinforcement learning (RL) is the dominant paradigm for sequential decision-making in AI research. We chose GOAP (Goal-Oriented Action Planning) backward-chaining instead for three reasons:

1. Cold start. RL requires thousands of episodes to learn a useful policy. GOAP produces valid plans from the first operator. After a single successful `cat` command during bootstrap, the Planner can chain it with `grep` to satisfy a `can_search_content` goal. RL would need to discover this composition through random exploration.

2. Interpretability. A GOAP plan is a sequence of operators with explicit preconditions and postconditions. It can be inspected, debugged, and validated *before execution*. An RL policy is a weight matrix — it cannot explain why it chose action a_t in state s_t ,

and its failures are opaque. For a self-modifying system, interpretability is not optional: the system must be able to reason about its own plans to improve them.

3. Compositionality. GOAP operators compose by postcondition matching. A new operator learned at time t is immediately available for plans at time $t + 1$, combined with operators learned at any prior time. RL policies are monolithic — they must be retrained from scratch when the action space changes. In NeuroSwarm, the action space changes *constantly* as new operators are learned, making RL retraining impractical.

The cost of GOAP is that postconditions must be inferred. RL learns implicit value functions from rewards; GOAP requires explicit semantic annotations. We solve this with pattern-based postcondition inference: `cat` $\rightarrow$ `can_read_file`, `g++` $\rightarrow$ `can_compile`, etc. This is imperfect but sufficient, and the imperfections are visible and fixable (unlike an RL policy’s implicit failures).

11.4 Why Genetic Variation Instead of Gradient Descent

The Variation Engine generates novel operators through mutation, recombination, and fragment assembly. This is an evolutionary algorithm, not a gradient-based optimiser. The reason is fundamental: **there is no differentiable objective function for shell commands**.

A shell command is a string that either succeeds or fails when executed. The fitness landscape is discrete, discontinuous, and non-smooth. Changing a single character can transform a valid command into a syntax error. Gradient descent requires a continuous, differentiable loss function that provides directional information (“move this parameter by δ to improve the objective by ϵ ”). Shell commands provide only binary feedback: exit code 0 or non-zero.

Evolutionary strategies are the only optimisation paradigm that operates on this type of landscape. They require only a fitness function (success/failure) and a variation operator (mutation/crossover). They are also naturally parallel: multiple candidates can be tested simultaneously in the Dream Sandbox.

The Variation Engine implements four strategies ordered by increasing randomness: template filling (most constrained), recombination (moderate), mutation (less constrained), and fragment assembly (least constrained). This ordering is deliberate: it produces candidates from the most promising region of the search space first, falling back to random exploration only when structured variation fails.

11.5 Why BK-tree + Levenshtein Instead of Neural Embeddings

For command validation, the obvious choice in 2026 is to embed commands with a transformer and measure cosine similarity. We chose Levenshtein distance with a BK-tree instead:

1. No GPU dependency. The BK-tree is a pure data structure — $O(n)$ insert, $O(n^\alpha)$ query (where $\alpha \approx 0.1\text{--}0.3$ empirically). It runs on any CPU with zero latency. Neural embeddings require a model loaded in VRAM (which is already saturated by the generative model) or a CPU inference pass that adds 50–200ms per query.

2. Deterministic and interpretable. Levenshtein distance returns an integer: “this command is 7 edits away from the nearest known-good command”. This is immediately interpretable and debuggable. Cosine similarity between embedding vectors returns a float whose meaning depends on the model, the tokenisation, and the training distribution. A distance of 0.73 might mean “very similar” or “completely different” depending on context.

3. Structural sensitivity. Levenshtein distance is sensitive to the syntactic structure of commands. `cat /etc/hosts` and `cat /etc/hostname` are distance 4 — structurally similar with a small path difference. A transformer embedding might place these at similar or dissimilar distances depending on whether `hosts` and `hostname` are in its vocabulary. For command validation, structural similarity (same verb, similar arguments) is exactly what matters.

4. Self-improving without retraining. The BK-tree grows with every successful command. Its discriminative power increases monotonically without any model update, retraining, or fine-tuning. It is an adaptive filter that requires zero maintenance.

The cost is that Levenshtein distance ignores semantic equivalence: `cat file.txt` and `less file.txt` are structurally distant but semantically identical. We accept this limitation because the validation gate is not the only filter — the Planner and ConceptLobe handle semantic equivalence at a higher level.

11.6 Why Local LLM Instead of API Calls

Every major AI agent framework (AutoGPT, LangChain, OpenAI Assistants, Claude Computer Use) depends on cloud API calls for inference. NeuroSwarm runs inference locally using llama.cpp with quantised models (Qwen2.5-7B, Q4_K_M, 4.4GB).

1. Autonomy. A system that depends on an external API is not autonomous — it is a client. If the API goes down, the system stops. If the API provider changes pricing, rate limits, or model behaviour, the system’s cognition changes without its knowledge or consent. An autonomous system must own its inference stack.

2. The LLM is a crutch, not a brain. In NeuroSwarm, the LLM is Tier 3 — the last resort after the Planner and Suggested Commands have failed. The explicit design goal is to **reduce LLM dependency over time**. Every successful LLM-generated command becomes a learned operator, available to the Planner for future goals. The LLM’s role monotonically shrinks. This is the opposite of API-based frameworks, where every action requires an API call regardless of experience.

3. Latency. A local 7B model generates tokens in 20–50ms on a consumer GPU. An API call requires 200–2000ms of network latency plus queue time. For a system that executes thousands of cognitive cycles per day, this difference compounds: local inference enables reactive behaviour that API latency makes impossible.

4. Privacy and security. NeuroSwarm executes shell commands on the host system. Sending command strings, file contents, and execution results to a cloud API creates an unacceptable security surface. Local inference keeps all data on the machine.

The cost is model quality. Qwen2.5-7B is vastly less capable than GPT-4 or Claude. The system compensates with the three-tier pipeline: it uses the weak model only when deterministic strategies fail, and it validates LLM output through the CriticLobe and BK-tree before execution. The weak model generates candidates; the rest of the architecture filters them.

11.7 Why Process-Per-Lobe Instead of Threads

Each lobe runs as a separate OS process, managed by the CerebralMatrix via `fork()/exec()`. The alternative — threads within a single process — would reduce memory overhead and eliminate IPC latency. We chose processes for one reason: **crash isolation**.

A segmentation fault in a thread kills the entire process. A segmentation fault in a child process kills only that child. The CerebralMatrix detects the crash via `waitpid()`, broadcasts a `lobe_crash` event, and restarts the lobe with exponential backoff. The rest of the system continues uninterrupted.

This is not a theoretical concern. During development, lobes crashed regularly due to malformed JSON, null pointer dereferences in edge cases, and ZMQ socket errors. With thread-based architecture, each crash would have killed the entire system. With process isolation, crashes are local events that trigger recovery, not catastrophe.

The CerebralMatrix also uses process isolation for **neurogenesis injection**. A new specialist lobe is a new executable — `fork()/exec()` is the natural mechanism. Injecting a new thread into a running process would require dynamic library loading (`dlopen`), symbol resolution, and careful state management. Process injection is trivially safe: the new process either starts successfully or it doesn't.

11.8 Why Not AutoGPT, LangChain, or Other Agent Frameworks

Existing LLM agent frameworks share a common architecture: a Python loop that sends prompts to a cloud API, parses the response, executes tools, and repeats. They differ in prompt engineering, tool selection, and memory management, but the fundamental structure is identical: **the LLM is the brain, and everything else is plumbing**.

NeuroSwarm inverts this relationship. The brain is the Planner + Basal Ganglia + Operator Registry — a deterministic, interpretable, self-improving system that requires no LLM. The LLM is a component that generates candidates when the deterministic system has no answer. This inversion has three consequences:

1. **Learning is structural, not prompt-based.** AutoGPT “learns” by appending past results to the prompt context window. When the window fills, old memories are lost. NeuroSwarm learns by adding operators to a persistent registry that survives across restarts, is searchable by postcondition, and is available to the Planner indefinitely.
2. **Performance improves without model changes.** Every successful execution reduces future LLM dependency. After 1000 executions, the Planner resolves the majority of goals without any inference call. AutoGPT's performance is bounded by the API model's capability — it cannot improve beyond what GPT-4 can generate.
3. **Self-modification is real, not simulated.** NeuroSwarm can generate, compile, and inject new C++ lobes into its own architecture. AutoGPT can write Python files, but it cannot modify its own execution loop, add new agent types, or change its tool-calling mechanism. The modification surface in NeuroSwarm is the entire system; in agent frameworks, it is limited to tool definitions.

The cost is complexity. NeuroSwarm is $\sim 15,000$ lines of C++ across 20+ source files. An equivalent AutoGPT configuration is ~ 200 lines of Python. The question is whether that complexity buys capabilities that simpler architectures cannot achieve. We argue that it does: no Python agent framework can detect a capability gap, generate a new module to address it, compile and inject that module at runtime, evaluate its performance, and terminate it if it degrades the system — all without human intervention or system restart.

Chapter 12

Discussion

12.1 Emergent Behaviours

Several behaviours emerge from the interaction of subsystems without being explicitly programmed:

- **Curiosity:** The fitness function naturally drives the system toward unknown domains. It explores not because it is told to, but because unexplored domains have high $N(d)$ and $P(d)$.
- **Learned helplessness:** When a domain fails repeatedly and the Basal Ganglia enters cooldown, the system temporarily avoids that domain — analogous to the psychological phenomenon of learned helplessness after repeated failure.
- **Skill transfer:** Operators learned in one domain become available to the Planner for goals in other domains. A `find` operator learned during `file_search` exploration can satisfy preconditions for `script_creation` goals.
- **Self-healing:** Crash detection + restart + neurogenesis means the system can recover from component failures and generate replacements for chronically dysfunctional subsystems.
- **Adaptive immune system:** The BK-tree command validation acts as a learned immune response. Initially permissive (cold start allows all commands), it becomes increasingly selective as the genome grows. Commands that are far from any previously successful command are rejected — analogous to the adaptive immune system recognising “non-self” molecules. The threshold adapts to command length, preventing both false positives (rejecting valid novel commands) and false negatives (accepting garbage).
- **Structural self-diagnosis:** MetaCognition’s knowledge gap graph creates a recursive model of the system’s own limitations. The precondition chain inference (“to write files, I need filesystem navigation, which requires resource discovery”) is a form of meta-level reasoning about capability dependencies — the system knows *why* it fails, not just *that* it fails.

12.2 LLM Dependency Trajectory

The architecture is designed so that LLM dependency monotonically decreases. Every successful LLM-generated command becomes a learned operator. Every learned operator is available to the Planner. The LLMOracle tracks a *dependency ratio*: $\frac{\text{total_calls}}{\text{total_calls}+100}$, which saturates as the denominator grows. In practice, after ~ 200 bootstrap cycles, the planner resolves the majority of goals without LLM involvement.

12.3 Limitations

- **Symbol grounding:** The system operates entirely within the symbolic domain of a filesystem. It has no sensorimotor grounding — no cameras, microphones, or actuators. This limits the depth of “understanding” it can achieve.
- **Single-machine bottleneck:** While the NetworkExpander can deploy kernels to remote hosts, inter-instance communication is limited to periodic SSH-based operator sync. There is no real-time distributed bus.
- **Operator quality:** Postcondition inference is pattern-based and imperfect. Complex multi-step commands may receive incorrect or missing postconditions, leading to suboptimal plans.
- **Genesis template rigidity:** The specialist lobe template is a fixed C++ skeleton. It cannot generate fundamentally novel lobe architectures — only domain-specific parameterisations of the same template.
- **No formal verification:** As the system rewrites its own code and generates new lobes, its behaviour becomes irreducibly emergent. Traditional testing frameworks cannot verify a system whose structure changes at runtime.

12.4 Relation to Theories of Consciousness

The architecture satisfies the structural prerequisites of two prominent theories:

Global Workspace Theory (Baars, 1988): The Thalamic bus implements a global workspace — information published by any lobe is accessible to all others. Attention (subscription filtering) determines which information reaches which modules.

Integrated Information Theory (Tononi, 2004): As the system generates new lobes and dense inter-lobe dependencies through neurogenesis and shared operator registries, its integrated information (Φ) necessarily increases. However, we make no claims about subjective experience — Φ measures structural integration, not phenomenal consciousness.

The system also instantiates a computational form of Friston’s Free Energy Principle: the Basal Ganglia explicitly minimises prediction error across capability domains, driving the system toward increasingly accurate world models through active inference.

Chapter 13

Conclusion

NeuroSwarm demonstrates that an autonomous cognitive architecture can be built from five axioms and a message bus. The system bootstraps from zero knowledge, learns operators through direct environmental interaction, plans using deterministic graph search, generates novel solutions through genetic variation, and extends its own anatomy when existing structures fail. The three-tier execution pipeline ensures that LLM inference is a last resort rather than the default mode of operation, and runtime learning continuously reduces LLM dependency.

The BK-tree command validation system introduces an adaptive quality gate between the LLM and execution: commands must be within Levenshtein distance of known-good commands to proceed. This filter is empty at birth and grows more discriminating with experience — a computational analogue of the adaptive immune system. Combined with genome compaction (clustering near-duplicates within distance 3), the system resists the degenerative feedback loop where bad LLM outputs pollute the genome and produce worse suggestions.

The recursive meta-cognition layer adds structural self-diagnosis: the system classifies its own failures into 16 error types, builds a knowledge gap graph with precondition chain inference, and directs exploration toward root causes. This transforms reactive error handling (“retry the same thing”) into proactive capability development (“learn filesystem navigation because that’s why file writes fail”).

The neurogenesis mechanism — where the Basal Ganglia writes C++ source, the MotorLobe compiles it, and the CerebralMatrix injects it as a live process — represents a concrete implementation of computational autopoiesis: the system literally produces its own components from within.

The architecture’s limitations — lack of sensorimotor grounding, template rigidity, absence of formal verification — point to clear future directions: robotic embodiment, meta-templates that generate templates, and runtime assertion frameworks for self-modifying systems. The question is no longer whether a system can autonomously modify its own structure, but how to reason about the behaviour of one that does.

Bibliography

- [1] Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.
- [2] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.
- [3] Harnad, S. (1990). The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42(1-3), 335–346.
- [4] Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing Company.
- [5] Revonsuo, A. (2000). The reinterpretation of dreams: An evolutionary hypothesis of the function of dreaming. *Behavioral and Brain Sciences*, 23(6), 877–901.
- [6] Schultz, W., Dayan, P., & Montague, P. R. (1997). A neural substrate of prediction and reward. *Science*, 275(5306), 1593–1599.
- [7] Tononi, G. (2004). An information integration theory of consciousness. *BMC Neuroscience*, 5(1), 42.
- [8] Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4), 189–208.
- [9] Orkin, J. (2006). Three states and a plan: the AI of F.E.A.R. *Game Developers Conference*.
- [10] Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- [11] Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence*, 47(1-3), 139–159.
- [12] Pfeifer, R., & Bongard, J. (2006). *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press.
- [13] Burkhard, W. A., & Keller, R. M. (1973). Some approaches to best-match file searching. *Communications of the ACM*, 16(4), 230–236.
- [14] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.